

VASP 장애 시 이벤트 수신 보장 — ChainEventListener 이중 경로

Webhook 한계 · ChainEventListener · 블록 스캔 복구
getLastProcessedBlock Phase 1 필수 구현

ChainEventListener

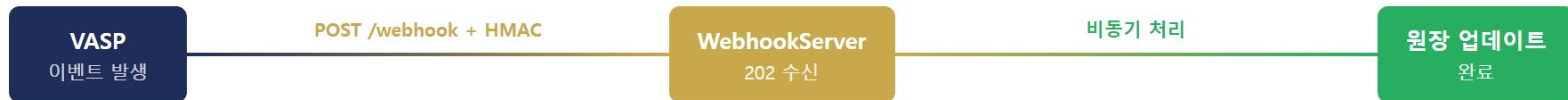
이중 경로 설계

Phase 1 필수 구현

Webhook의 구조적 한계 — at-most-once

WEBHOOK RELIABILITY PROBLEM — DELIVERY IS NEVER GUARANTEED

정상 흐름



장애 시나리오



결론 — Webhook은 본질적으로 at-most-once. VASP가 재시도하더라도 우리가 그 정책을 제어할 수 없다.

202 즉시 응답 설계의 트레이드오프

ACCEPT-BEFORE-PROCESS — SPEED VS DURABILITY

코드 — WebhookServer.ts 107번째 줄

```
// ① 처리 전에 먼저 응답  
res.writeHead(202).end();  
  
// ② 이후 비동기 처리 (메모리에서)  
await Promise.allSettled(  
  handlers.map(h => h(payload))  
);
```

①과 ② 사이 크래시가 발생하면

VASP는 이미 202를 받았으므로 "전달 완료"로 판단 →

payload는 메모리에서 사라지고 **복구 방법 없음**

✅ 장점

- VASP timeout 방지 — 빠른 확인 응답
- VASP 재시도 루프 차단
- 처리 지연이 발신자에게 전파되지 않음

❌ 단점

- 202 후 크래시 시 payload 영구 유실
- Redis-DB persist 없이는 복구 불가
- at-most-once 구조 벗어날 수 없음

완전한 해결책 — 202 전에 Redis XADD로 먼저 저장하는 Transactional Inbox 패턴. 현재는 ChainEventListener가 이 역할을 대신.

해법 — ChainEventListener 이중 경로

CHAIN EVENT LISTENER AS RELIABLE BACKUP PATH

이중 경로 구조

경로 A (빠른 알림) VASP → Webhook → WebhookServer → 즉시 처리

정상 시: 수 초 내 처리. VASP 장애 시: 유실 위험

경로 B (보장 경로) 블록체인 RPC → ChainEventListener → 처리

VASP 독립. 블록이 존재하는 한 반드시 수신

핵심 통찰 — 블록체인은 영구 로그. VASP가 알려주지 않아도 RPC 노드에서 직접 읽으면 된다.

ChainEventListener 구현 (index.ts)

```
const eventListener = new ChainEventListener(
  chainAdapter,
  [nftIssuedHandler],
  [{
    addr:      process.env.NFT_CONTRACT_ADDR!,
    abi:        NFTIssuerABI,
    eventNames: ['Issued'],
  }],
  {
    // Phase 2: 마지막 처리 블록 저장
    async getLastProcessedBlock() {
      return 0; // ← Phase 1 임시
    },
    async setLastProcessedBlock(_b) {},
  },
);
```

멥등성 보장 — 같은 이벤트를 Webhook과 ChainEventListener 양쪽에서 받아도 IdempotencyGuard가 중복 처리를 차단.

Webhook vs ChainEventListener — 비교

TWO PATHS COMPARED — SPEED VS RELIABILITY

항목	경로 A — VASP Webhook	경로 B — ChainEventListener
이벤트 소스	VASP (외부 서비스)	블록체인 RPC 노드 (직접)
전달 보장	At-most-once (최대 한 번) — VASP 정책에 의존	At-least-once (최소 한 번) — 블록 영구 보존
지연	수 초 — VASP가 즉시 알림	블록 주기 — 폴링 간격에 따라
VASP 장애 영향	이벤트 유실	영향 없음 — RPC 직접 접근
우리 서버 재시작 시	미수신 이벤트 복구 불가	마지막 블록부터 재스캔
중복 수신 가능성	VASP 재시도 시 발생	재시작 시 발생
중복 처리 방어	IdempotencyGuard (txHash + logIndex) — 양쪽 공통 방어선	

설계 원칙 — 경로 A는 속도, 경로 B는 보장. 두 경로가 같은 이벤트를 각자 독립적으로 처리하되, 멍등성이 중복을 막는다. 이것이 at-least-once + 멍등성의 확장 적용.

getLastProcessedBlock — Phase 1 필수 구현

BLOCK CURSOR PERSISTENCE — REQUIRED FOR VASP FALLBACK TO WORK

현재 stub — ChainEventListener 안전망이 무력화됨

```
// index.ts - 현재 stub 구현
{
  async getLastProcessedBlock() {
    return 0; // 항상 genesis부터 스캔
  },
  async setLastProcessedBlock(_b) {
    // 아무것도 저장 안 함
  },
}
```

핵심 문제 — VASP 장애로 웹훅이 유실됐을 때 ChainEventListener가 안전망 역할을 해야 하는데, 재시작 시 블록 0부터 수백만 블록 재스캔 → RPC 과부하-타임아웃으로 정작 필요한 순간에 실패할 수 있다.

결론 — VASP 단독으로 수신 보장이 안 되고, ChainEventListener도 블록 커서 없이는 보장 안 됨. 두 개는 세트.

필수 구현 — Redis에 블록 커서 저장

```
// Phase 1 필수 구현
{
  async getLastProcessedBlock() {
    const val = await redis.get(
      'kyobo:last-block'
    );
    return val ? parseInt(val, 10) : 0;
  },
  async setLastProcessedBlock(b) {
    await redis.set(
      'kyobo:last-block',
      String(b)
    );
  },
}
```

효과 — 재시작 시 마지막 처리 블록 이후만 스캔. VASP 장애 구간의 이벤트를 정확히 복구. ChainEventListener가 진짜 안전망으로 작동.

정리 — VASP 이벤트 수신 신뢰성 전체 그림

SUMMARY — DUAL PATH RELIABILITY DESIGN

① VASP Webhook (경로 A)

- 정상 시 수 초 내 빠른 처리
- HMAC 서명으로 위변조 방어
- IdempotencyGuard로 중복 차단
- VASP 장애 시 이벤트 유실 위험
- at-most-once — 보장 없음

② ChainEventListener (경로 B)

- 블록체인 직접 폴링 — VASP 독립
- 블록은 영구 저장 — 절대 유실 없음
- 재시작 후 블록 커서부터 복구
- at-least-once 보장
- 블록 커서 persist — Phase 1 필수

③ 공통 방어선

- IdempotencyGuard
- 멍등키: txHash + logIndex
- 양쪽 경로 동시 수신 → 중복 무시
- 중복 처리 100% 차단
- 원장 정합성 유지

결론 — VASP 장애가 발생해도 ChainEventListener가 블록을 직접 스캔해 이벤트를 복구한다. 두 경로가 독립적으로 동작하고 멍등성이 중복을 막는 구조 — 이것이 이 시스템의 신뢰성 근거.

Phase 1 필수 액션 — getLastProcessedBlock / setLastProcessedBlock을 Redis 구현체로 교체. stub 상태에서는 VASP 안전망(ChainEventListener)이 제 역할을 못 한다. VASP 보장 + 블록 커서가 세트로 동작해야 수신 신뢰성이 완성된다.